DAY 11 TECHNICAL ACTIVITY REPORT

Introduction to Databases & CRUD Architecture

Transitioning from Volatile In-Memory Storage to Persistent Relational Databases

TRAINEE	MENTOR	INSTITUTION	DATE & STATUS
Benatic Rithan B	Rathees G	KIT, Coimbatore Dept of CSE	20 June 2026 ● Completed Successfully

1. EXECUTIVE SUMMARY

This technical report documents the design and execution parameters verified during Day 11 of the Backend Development track. The foundational goal of this session was to transition data persistence lifecycle layers away from temporary in-memory entities and establish a durable, relational environment using MySQL server architectures. An enterprise-styled `Employee Database` was built to enforce key validation constraints and execute standard Create, Read, Update, and Delete (CRUD) operations safely while ensuring system data integrity.

2. DATA PERSISTENCE ARCHITECTURE & COMPLEXITY LOGIC

To prevent structural data loss common in volatile program states, core memory paradigms were evolved into disk-based relational storage vectors:

- **Persistent Storage Arrays:** Bypasses runtime server dependencies by saving transaction items to disk blocks rather than ephemeral virtual environments.
- **Primary Index Tuning:** Assigns strict unique indexes to speed up table scans from linear checks down to isolated constant point evaluations, $O(1)$.
- **Declarative Integrity Filtering:** Uses structural query logic strings to process entity evaluations instead of executing continuous loop transformations across collections.

3. STRUCTURAL MAPPING MATRIX

CRUD OPERATION	SQL ELEMENT ROLE	CORE RELATIONAL ACTION	PRIMARY OPERATIONAL ADVANTAGE
Create	`INSERT INTO`	Appends a structured row array into the targeted database map leaves.	Enforces strict column types on entry instantiation.
Read	`SELECT ... WHERE`	Queries specific storage lines by indexing exact identifier matching terms.	Avoids costly table scans across large file structures.
Update	`UPDATE ... SET`	Modifies field parameters in-place inside target record addresses.	Changes targeted cells without rewriting the surrounding schema rows.
Delete	`DELETE FROM`	Cleans tracking blocks by purging unneeded rows from catalog systems.	Reclaims file space safely while protecting schema boundaries.

4. TECHNICAL CODE IMPLEMENTATION

The script segment below details the clean SQL architecture engineered to deploy the core relational schema along with functional constraints for data security:

```sql
-- Initialize Schema Isolation Domain
CREATE DATABASE IF NOT EXISTS EnterpriseDB;
USE EnterpriseDB;

-- Construct Tabular Map with Attribute Rules
CREATE TABLE employees (
    employee_id INT AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    department VARCHAR(50) DEFAULT 'General Operations',
    salary DECIMAL(10, 2) CHECK (salary > 0),
    hire_date DATE NOT NULL,
    PRIMARY KEY (employee_id)
);

-- Execute Verified CRUD Transactions
INSERT INTO employees (first_name, last_name, department, salary, hire_date)
VALUES ('Benatic', 'Rithan', 'Computer Science Engineering', 85000.00, '2026-06-12');

SELECT * FROM employees WHERE employee_id = 1;

UPDATE employees SET salary = 92000.00 WHERE employee_id = 1;
```

5. WORKSPACE ENVIRONMENT STATE MOCKUPS

MYSQL RELATIONAL TABLE LAYOUT (EMPLOYEES ACTIVE VIEW)

```
+-------------+------------+-----------+------------------------------+-----------+------------+
| employee_id | first_name | last_name | department                   | salary    | hire_date  |
+-------------+------------+-----------+------------------------------+-----------+------------+
| 1           | Benatic    | Rithan    | Computer Science Engineering | 92000.00  | 2026-06-12 |
| 2           | John       | Doe       | General Operations           | 60000.00  | 2026-06-15 |
+-------------+------------+-----------+------------------------------+-----------+------------+
2 rows in set (0.00 sec)
```

VERSION CONTROL TRACK METRICS (GITHUB SUBMISSION REPOSITORY STATE)

```
Origin: remote upstream master branch
[Commit Ref: #DB-D11-S10] feat: deploy local MySQL relational database configurations
[Commit Ref: #DB-D11-S11] feat: construct structured employee tables with incremental indexes
[Commit Ref: #DB-D11-S12] docs: verify standard CRUD script logs against enterprise schema
```

6. LEARNING OUTCOMES

- **Relational Schema Conception:** Architected logical relational tables complete with strict parameter rules, type controls, and dedicated primary tracking identities.
- **Local RDBMS Infrastructure Configuration:** Deployed, initialized, and secured a local MySQL server runtime instance safely across host workspace environments.
- **CRUD Transaction Execution:** Formulated precise structured statements to create, extract, alter, and delete operational data lines cleanly without altering adjacent records.

- **Constraint Enforcement:** Configured strict constraint attributes to protect storage schemas against faulty inputs or incorrect structural types.